

Relatório Técnico: Classificação de Pneumonia em Raio-X

Daniel Vaz Barbosa

Kauê Barbosa Oliveira

13 de janeiro de 2026

Resumo

Este relatório descreve o desenvolvimento de um pipeline computacional para a detecção automática de pneumonia em imagens de Raio-X do tórax. A abordagem utilizada baseia-se na extração de características de textura (GLCM) e estatísticas de intensidade, evitando o uso de redes neurais profundas em favor de modelos clássicos otimizados. O projeto foi estruturado em scripts modulares para análise exploratória, extração de atributos e otimização de hiperparâmetros via *Grid Search*. O modelo final, uma Regressão Logística com normalização Min-Max, atingiu uma acurácia de 78,21% e alta sensibilidade na detecção da doença.

Conteúdo

1	Introdução	2
2	Metodologia e Implementação	2
2.1	Análise Exploratória (<code>load_dataset.py</code>)	2
2.2	Pré-processamento e Extração de Atributos (<code>feature_extraction.py</code>) . .	3
2.2.1	Tratamento de Imagem	3
2.2.2	Mineração de Atributos (GLCM)	3
2.3	Classificação Baseline (<code>classification_metrics.py</code>)	3
2.4	Otimização Avançada (<code>gridsearch.py</code>)	4
3	Resultados	4
3.1	Comparativo de Modelos	4
3.2	Análise do Modelo Campeão	5
4	Conclusão	5

1 Introdução

A pneumonia é uma infecção respiratória grave que requer diagnóstico rápido e preciso. O objetivo deste projeto foi desenvolver um sistema de suporte à decisão clínica capaz de diferenciar imagens de Raio-X do tórax em duas classes: **Normal** e **Pneumonia**.

A metodologia adotada priorizou a interpretabilidade e a eficiência computacional, utilizando técnicas de Processamento Digital de Imagens para extração de atributos e algoritmos de aprendizado de máquina supervisionado disponíveis na biblioteca **Scikit-Learn**.

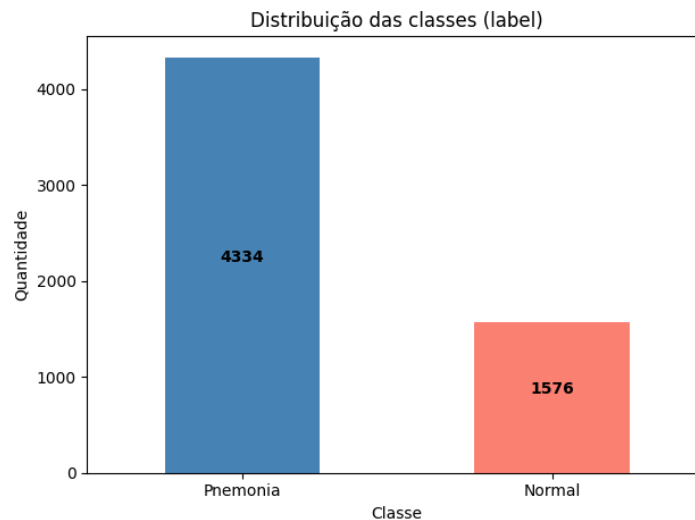
2 Metodologia e Implementação

O desenvolvimento foi segmentado em quatro etapas lógicas, correspondentes aos scripts Python desenvolvidos para o projeto.

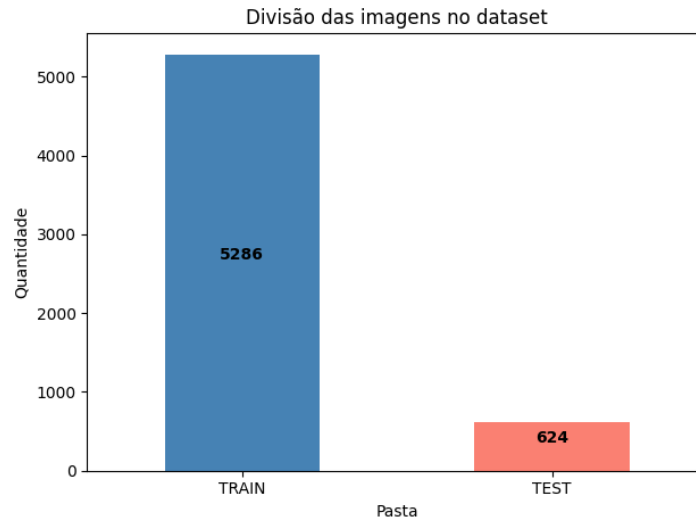
2.1 Análise Exploratória (`load_dataset.py`)

Nesta etapa inicial, foi realizada a leitura dos metadados (`Chest_xray_Corona_Metadata.csv`) para compreender a estrutura do *dataset*. As principais ações realizadas por este script incluíram:

- **Verificação de Classes:** Identificação de um desequilíbrio significativo, com maior prevalência de casos de Pneumonia em relação a casos Normais.



- **Particionamento:** Confirmação da divisão dos dados em pastas de Treino (TRAIN) e Teste (TEST), fundamental para garantir a integridade da avaliação e evitar vazamento de dados (*data leakage*).



2.2 Pré-processamento e Extração de Atributos (feature_extraction.py)

Este script constituiu o núcleo da engenharia de características, transformando as imagens brutas em dados tabulares estruturados (`atributos_train.csv` e `atributos_test.csv`).

2.2.1 Tratamento de Imagem

- **Escala de Cinza:** Conversão utilizando `cv2.IMREAD_GRAYSCALE` para eliminar informações de cor e focar na luminância e densidade dos tecidos.
- **Redimensionamento:** Aplicação de `cv2.resize` para 128x128 pixels. Este processo padronizou a escala para análise de textura e atuou como um filtro passa-baixa, reduzindo ruídos de alta frequência inerentes à captura do exame.

2.2.2 Mineração de Atributos (GLCM)

Utilizando a biblioteca `skimage.feature`, calculou-se a Matriz de Coocorrência de Níveis de Cinza (GLCM) com distância=1 e ângulo=0°. Foram extraídos 6 atributos por imagem:

Tabela 1: Atributos Extraídos

Categoria	Atributos e Descrição
Textura	Contraste, Homogeneidade, Energia, Correlação. Capturam a rugosidade e padrões espaciais do tecido pulmonar.
Estatística	Média de Pixels, Desvio Padrão. Indicam a intensidade global (pulmões com pneumonia tendem a ser mais opacos/brancos).

2.3 Classificação Baseline (classification_metrics.py)

Para estabelecer uma linha de base, implementou-se um modelo inicial utilizando `RandomForestClassifier`. Embora funcional, este modelo inicial revelou um viés para

a classe majoritária (Pneumonia), evidenciando a necessidade de normalização dos dados, visto que atributos como "Contraste" possuem escalas numéricas muito superiores a atributos como "Energia".

2.4 Otimização Avançada (gridsearch.py)

O script final implementou uma busca exaustiva pelos melhores hiperparâmetros (*Grid Search*), corrigindo as limitações do modelo baseline.

- **Pipeline:** Criação de um fluxo automatizado contendo `MinMaxScaler` (normalização entre 0 e 1) seguido do classificador.
- **Modelos Avaliados:** Logistic Regression, SVM, Random Forest, KNN e Gradient Boosting.
- **Validação:** Utilização de *Cross-Validation* (cv=3) e balanceamento de classes (`class_weight='balanced'`).

3 Resultados

A avaliação final foi realizada no conjunto de teste independente, composto por dados nunca vistos durante o treinamento.

3.1 Comparativo de Modelos

A Tabela 2 apresenta o desempenho dos algoritmos após a otimização.

Tabela 2: Ranking de Acurácia no Conjunto de Teste

Modelo	Acurácia	Observação
Logistic Regression	78,21%	Melhor estabilidade e generalização
SVM (Polinomial)	77,08%	Alto custo computacional
Random Forest	76,92%	Leve <i>overfitting</i> comparado ao treino
KNN	76,28%	Prejudicado pela dimensionalidade
Gradient Boosting	76,12%	Desempenho inferior neste cenário

3.2 Análise do Modelo Campeão

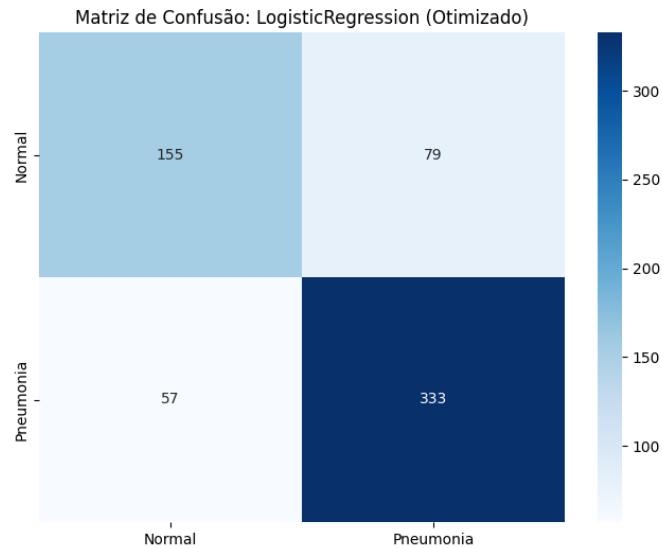


Figura 1: Matriz de Confusão do Modelo Otimizado

O modelo de **Regressão Logística** (com *solver* ‘liblinear’ e regularização $C=100$) demonstrou ser a solução mais robusta.

- **Acurácia Global:** 78,21%
- **Sensibilidade (Recall) para Pneumonia:** 85%
- **Sensibilidade (Recall) para Normal:** 66%

A alta sensibilidade para a classe Pneumonia indica que o sistema é eficaz em evitar falsos negativos, característica crítica para triagem médica. A aplicação do **MinMaxScaler** foi decisiva para este resultado, pois colocou atributos de grandezas dispares (como Contraste e Energia) na mesma magnitude, permitindo a convergência ideal do algoritmo linear.

4 Conclusão

A implementação modular através dos scripts permitiu isolar e validar cada etapa do processo de Ciência de Dados. A transição de um modelo de árvore simples (`classification_metrics.py`) para um pipeline otimizado com normalização (`gridsearch.py`) resultou em um ganho de performance e confiabilidade.

Conclui-se que, para o conjunto reduzido de atributos extraídos via GLCM, a Regressão Logística com normalização Min-Max é a solução mais eficiente, oferecendo um equilíbrio ideal entre acurácia, interpretabilidade e custo computacional.